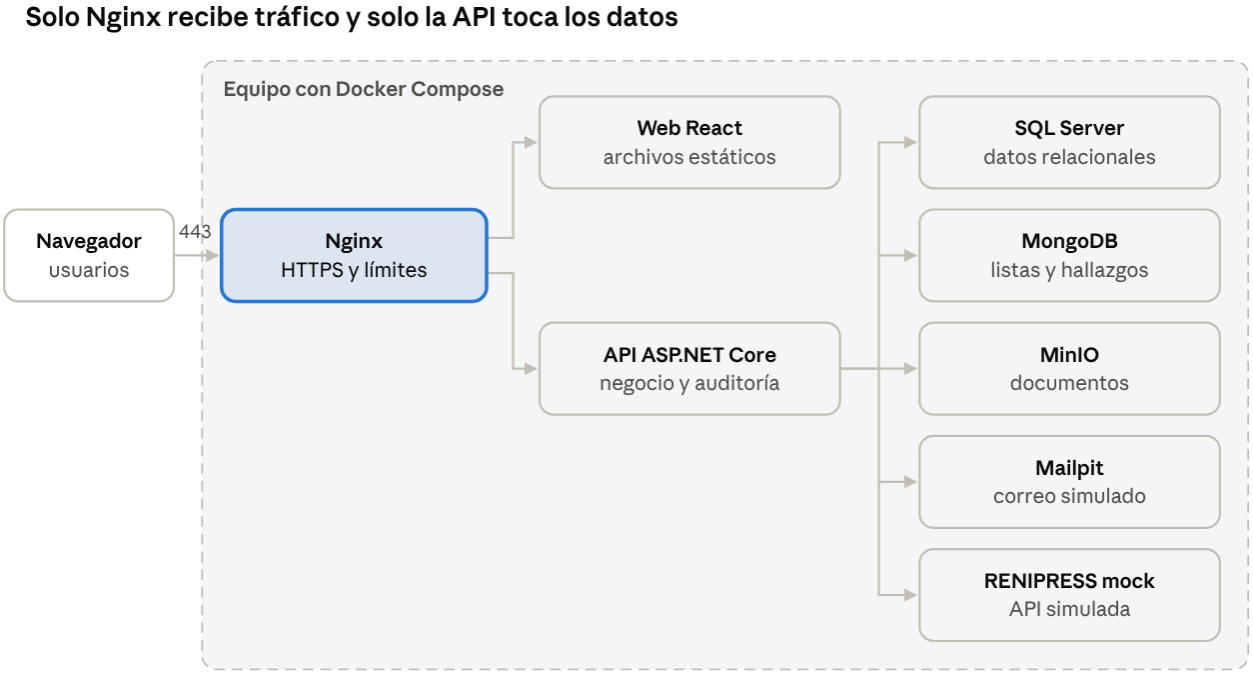
Guía de despliegue y operaciones – Sistema de Autorización Sanitaria

El sistema completo se levanta con Docker Compose en un solo equipo: ocho contenedores, de los cuales solo Nginx recibe tráfico externo.

1. Arquitectura de despliegue



Arquitectura de despliegue · 8 contenedores

Nginx es la única entrada (capa de redirección y seguridad); las tres bases y los dos servicios simulados solo son accesibles desde la API, dentro de la red interna de Docker.

2. Requisitos previos y puertos

Software en el equipo:

- Docker Desktop (Windows o macOS) o Docker Engine con el plugin Compose (Linux)
- Git
- OpenSSL, para generar el certificado local (viene con Git para Windows)
- Al menos 8 GB de RAM; SQL Server necesita 2 GB para sí mismo
- Solo para desarrollar fuera de Docker: .NET 10 SDK y Node.js LTS

Contenedores y puertos:

Contenedor	Imagen	Puerto interno	Expuesto al equipo	Uso
nginx	nginx:stable-alpine	80, 443	80, 443	Entrada única, HTTPS, límite de peticiones
web	build de `frontend/`	80	No	Archivos estáticos de React
api	build de `src/`	8080	No	ASP.NET Core Web API
sqlserver	mcr.microsoft.com/mssql/server:2022-latest	1433	1433 (solo desarrollo)	Base de datos relacional
mongo	mongo:7	27017	27017 (solo desarrollo)	Listas de verificación y hallazgos
minio	minio/minio	9000, 9001	9001 (consola)	Almacenamiento de documentos
mailpit	axllent/mailpit	1025, 8025	8025 (bandeja web)	SMTP simulado para notificaciones
renipress-mock	build de `src/RenipressMock/`	8080	No	API RENIPRESS simulada

3. Configuración

Toda la configuración está en la carpeta `deploy/` del repositorio. Antes de desplegar solo hay que completar las variables de entorno; los demás archivos ya vienen listos.

3.1 Variables de entorno

Se copian de `deploy/.env.example` a `deploy/.env`. Este archivo no se sube a GitHub, porque contiene las contraseñas (RNF-26).

Variable	Para qué sirve	Regla
MSSQL_SA_PASSWORD	Contraseña del administrador de SQL Server	Mínimo 8 caracteres con mayúscula, minúscula, número y símbolo
MONGO_USER, MONGO_PASSWORD	Usuario administrador de MongoDB	Contraseña distinta a la de SQL Server

Variable	Para qué sirve	Regla
MINIO_ROOT_USER, MINIO_ROOT_PASSWORD	Acceso a MinIO	Contraseña de al menos 8 caracteres
MINIO_BUCKET	Nombre del espacio donde se guardan los documentos	Por defecto: `documentos`
JWT_KEY	Clave para firmar los tokens de sesión	Al menos 32 caracteres aleatorios
JWT_MINUTES	Duración del token de sesión	30 (RNF-02)
ADMIN_EMAIL, ADMIN_PASSWORD	Primer usuario administrador	Cambiar la contraseña tras el primer ingreso

3.2 Servicios y datos persistentes

El archivo `deploy/docker-compose.yml` define los ocho contenedores de la sección 2 y cómo se conectan. Los datos se guardan en tres volúmenes de Docker, que sobreviven cuando los contenedores se reinician o se recrean:

Volumen	Qué guarda
sql-data	Base de datos de SQL Server
mongo-data	Listas de verificación, actas y hallazgos
minio-data	Planos, fotografías, certificados, informes y resoluciones

SQL Server (1433) y MongoDB (27017) se exponen al equipo solo para revisarlos durante el desarrollo; en un servidor real esos puertos se cierran.

3.3 Proxy inverso (Nginx)

El archivo `deploy/nginx/nginx.conf` configura la única entrada al sistema:

- Redirige todo el tráfico HTTP a HTTPS, con el certificado de `deploy/nginx/certs/`.
- Envía las rutas `/api/` a la API y el resto a la aplicación web.
- Rechaza archivos de más de 20 MB (RNF-15).
- Limita el inicio de sesión a 10 peticiones por segundo por IP (RNF-07).

- Asigna a cada petición un identificador `X-Request-Id`, que aparece en los logs de la API (RNF-12).

4. Pasos de despliegue

Desde cero, el despliegue toma unos 10 minutos; la primera vez tarda más por la descarga de imágenes.

1. **Descargar el proyecto.** Clonar el repositorio desde GitHub y entrar a la carpeta `deploy/`.
2. **Configurar las contraseñas.** Copiar `.env.example` como `.env` y cambiar todas las contraseñas (sección 3.1).
3. **Generar el certificado HTTPS.** Crear un certificado autofirmado con OpenSSL y guardarlo en `nginx/certs/` como `cert.pem` y `key.pem`. En un servidor real se usa un certificado emitido por una autoridad.
4. **Levantar el sistema.** Ejecutar `docker compose up -d --build`. La primera vez tarda más porque descarga las imágenes.
5. **Comprobar los contenedores.** Ejecutar `docker compose ps` y verificar que los ocho estén en estado `running`.
6. **Preparación automática.** Al iniciar, la API crea las tablas de la base de datos, el espacio de documentos en MinIO, los catálogos iniciales y el usuario administrador. No hay que hacerlo a mano.
7. **Verificar el sistema** con la tabla siguiente.

Qué se verifica	Dónde	Resultado esperado
Aplicación web	https://localhost	Pantalla de inicio de sesión (aceptar el aviso del certificado local)
Salud de la API	https://localhost/api/health	`Healthy`
Inicio de sesión	Con `ADMIN_EMAIL` y `ADMIN_PASSWORD`	Acceso al panel del administrador
Notificaciones	http://localhost:8025	Bandeja de Mailpit con los correos enviados
Documentos	http://localhost:9001	Consola de MinIO con el bucket `documentos`

5. Operación

Todos los comandos se ejecutan desde la carpeta `deploy/`.

5.1 Logs

La API escribe logs en JSON con Serilog; cada línea incluye el `RequestId` que Nginx asigna.

Para qué	Comando
Ver los logs de la API en tiempo real	`docker compose logs -f api`
Rastrear una petición	Buscar su `RequestId` en los logs de la API
Ver accesos y errores del proxy	`docker compose logs nginx`

La bitácora de auditoría (quién hizo qué) no está en estos logs: se consulta en la aplicación, con CUS-22.

5.2 Monitoreo

Qué revisar	Cómo	Señal de alerta
Estado de los contenedores	`docker compose ps`	Alguno en `exited` o `restarting`
Salud de la API y sus dependencias	https://localhost/api/health	`Unhealthy` o `Degraded`
Métricas (peticiones, tiempos, errores)	https://localhost/api/metrics	Errores 5xx en aumento
Uso de recursos	`docker stats`	SQL Server sobre 90% de su memoria

5.3 Respaldo diario

Las tres bases se respaldan juntas una vez al día (RNF-18). Lo hace el script `deploy/backup.sh`, que se programa en el equipo para que corra solo y guarda todo en `deploy/backups/<fecha>/`:

Qué se respalda	Herramienta	Archivo generado
Base de datos SQL Server	Comando BACKUP DATABASE de SQL Server	`as.bak`
Colecciones de MongoDB	`mongodump`	`mongo.archive`
Documentos de MinIO	Cliente `mc` de MinIO (copia espejo)	Carpeta `minio/`

Para respaldar manualmente, por ejemplo antes de una actualización, se ejecuta `./backup.sh` desde la carpeta `deploy/`. Se recomienda guardar una copia de los respaldos fuera del equipo.

5.4 Restauración

Se restaura desde la carpeta de respaldo de la fecha que se quiere recuperar:

1. Detener la API con `docker compose stop api`, para que nadie modifique datos durante la restauración.
2. Restaurar SQL Server desde `as.bak` con el comando RESTORE DATABASE, reemplazando la base actual.
3. Restaurar MongoDB desde `mongo.archive` con `mongorestore`, reemplazando las colecciones actuales.
4. Restaurar los documentos copiando la carpeta `minio/` de vuelta a MinIO con el cliente `mc`.
5. Iniciar la API con `docker compose start api` y comprobar `/api/health`.

El script `deploy/restore.sh <fecha>` ejecuta estos pasos en orden.

5.5 Actualización a una nueva versión

Primero se hace un respaldo. Luego se descarga la nueva versión con `git pull` y se reconstruyen la API y la web con `docker compose up -d --build api web`.

Las migraciones nuevas se aplican solas al reiniciar la API. Antes de actualizar, ejecutar el respaldo.

5.6 Detener el sistema

- `docker compose down` detiene y elimina los contenedores, pero **conserva los datos** (volúmenes).

- `docker compose down -v` **borra también los datos**. Usar solo para reiniciar desde cero.

6. Problemas frecuentes

Síntoma	Causa probable	Solución
`sqlserver` se detiene al iniciar	Contraseña SA débil o menos de 2 GB de RAM para Docker	Usar una contraseña compleja en `.env`; subir la memoria de Docker Desktop
La API falla con error de conexión a SQL Server	SQL Server aún no termina de iniciar	Esperar unos segundos y ejecutar `docker compose restart api`
El puerto 80 o 443 está ocupado	Otro servidor web en el equipo (IIS, Apache)	Detener ese servicio o cambiar el puerto expuesto en `docker-compose.yml`
El navegador muestra “conexión no segura”	Certificado autofirmado local	Es esperado en desarrollo; aceptar el aviso
Error 413 al subir un documento	Archivo mayor de 20 MB	Reducir el archivo; el límite es intencional (RNF-15)
Error 503 en el login	Se superó el límite de peticiones	Esperar un momento; es la protección de RNF-07
No llegan notificaciones	Mailpit detenido o mal configurado `Smtp__Host`	Revisar `docker compose ps` y la bandeja en http://localhost:8025
Se perdieron los datos tras reiniciar	Se usó `docker compose down -v`	Restaurar desde el último respaldo (sección 5.4)